ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled "**REAL TIME CHAT APPLICATION**" is submitted by

GOROGODO TAKUNDA LEONARD	(24B11CS488)
M NAGA ABHILASH	(24B11CS267)
NYAMHAMBA BUDWELL K	(24B11CS511)
KARRI LOKESH	(25B21CS080)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Information Technology for the academic year 2025-2026.

Project Guide
Mrs.R.Padma Sri,
Assistant Professor,
Department of CSE

Head of the Department
Dr.T.Sudha Rani, Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled "**REAL TIME CHAT APPLICATION**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Computer Science and Engineering.** We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place:Suramm

palemDate:

By

GOROGODO TAKUNDA LEONARD	(24B11CS488)
M NAGA ABHILASH	(24B11CS267)
NYAMHAMBA BUDWELL K	(24B11CS511)
KARRI LOKESH	(25B21CS080)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our **REAL TIME CHAT APPLICATION** Cornerstone Project Guide, **Mrs.R.Padma Sri, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr.K.Chandra Sekhar, Associate Professor and Dr.Hari Jyothula, Assistant Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr.T.Sudha Rani, Associate Professor,**for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree , Pro Vice-Chancellor (Academics), Dr . M.B. Srinivas, Vice -Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of Contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURE	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	4
2.3.1 Minimum Hardware Requirements	4
2.3.2 Software Requirements	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Language	5
3.1.1 Introduction to JavaScript	5
3.2 Framework / Libraries	5
3.2.1 Introduction to React	5

3.2.2 Introduction to Node.js 5

3.2.3 Introduction to Socket.IO 5

3.3 Database Technology 5

3.3.1 MongoDB 6

3.4 Tools & IDEs 6

3.4.1 Visual Studio Code 6

4. DATABASE DESIGN 7

4.1 Database Collections 7

4.2 Collection Structure 7

4.3 Keys and Constraints 7

4.4 Collection Relationships 8

5. SYSTEM DESIGN 9

5.1 System Architecture 9

5.2 Modules Description 10

6. SYSTEM IMPLEMENTATION 11

7. CONCLUSION 15

8. FUTURE ENHANCEMENTS 16

9. REFERENCES 17

APPENDIX (Sample Code) 18

APPENDIX (API Endpoints) 19

Abstract

The Real-Time Chat Application is a web-based messaging platform designed to enable instant communication between users through a modern, responsive interface. The system provides one-to-one messaging capabilities with real-time message delivery using WebSocket technology, ensuring messages appear instantly without requiring page refresh. Users can send text messages, emojis, and view message timestamps with delivery and read status indicators.

The application features presence detection that shows which users are currently online or offline, creating a more engaging communication experience. Developed using the MERN stack (MongoDB, Express.js, React, Node.js) with Socket.IO for real-time functionality, the system ensures reliable message delivery and data persistence. With its intuitive interface and efficient real-time communication capabilities, the application provides a robust solution for instant messaging needs.

List of Figures

Figure No.	Figure Title	Page No.
1	Login Screen of Real-Time Chat Application	11
2	Chat Interface Screen of Real-Time Chat Application	12
3	Message Sending Screen of Real-Time Chat Application	13
4	Online Users Screen of Real-Time Chat Application	13
5	Message Status Screen of Real-Time Chat Application	14
6	User Profile Screen of Real-Time Chat Application	14

List of Tables

Table No.	Table Title	Page No.
1	Users Collection Structure	7
2	Messages Collection Structure	7

Acronyms

- **RTCA** : Real-Time Chat Application
- **API** : Application Programming Interface
- **UI** : User Interface
- **UX** : User Experience
- **HTTP** : HyperText Transfer Protocol
- **HTTPS** : HyperText Transfer Protocol Secure
- **JSON** : JavaScript Object Notation
- **JWT** : JSON Web Token

1.Introduction

1.1 Brief Information about the Project

The Real-Time Chat Application (RTCA) is a web-based messaging platform developed to facilitate instant communication between users. It provides a centralized system for sending and receiving messages in real-time without requiring manual page refresh. The application replaces traditional email and delayed messaging systems with instant, WebSocket-based communication.

The system is built using modern web technologies including React for the frontend interface, Node.js and Express.js for the backend server, MongoDB for data storage, and Socket.IO for real-time messaging functionality. It features user authentication, message persistence, online presence detection, and emoji support. The application is designed to be responsive and accessible across various devices including desktops, tablets, and smartphones.

1.2 Motivation and Contribution of Project

Traditional messaging systems often require users to refresh pages or wait for periodic server polling to receive new messages, creating delays and poor user experience. Many existing solutions lack presence detection or message status indicators, making it difficult to know if messages were delivered or read.

The key contributions of this project include:

- Real-time message delivery using WebSocket technology
- Instant presence detection showing online/offline user status
- Message delivery and read confirmation indicators
- Emoji support for enhanced communication
- Message timestamp tracking
- Secure user authentication and data protection

This system simplifies instant communication and improves overall messaging efficiency.

1.3 Objectives of the Project

The main objectives of the Real-Time Chat Application are:

- To develop a responsive application for real-time messaging
- To store user and message data in a structured database
- To provide functionalities for sending, receiving, and viewing messages
- To enable presence detection showing online/offline status
- To display message timestamps and delivery/read status
- To support emoji sending and rendering

1.4 Scope of the Project

The Real-Time Chat Application can be used in various environments including educational institutions, corporate offices, and personal communication. It supports operations such as user registration and login, one-to-one messaging, emoji sending, viewing online users, and tracking message status. The system provides a simple and intuitive interface for maintaining conversations in a centralized platform.

2.System Analysis

2.1 Functional Requirements

Functional requirements define the core operations that the Real-Time Chat Application must perform:

- **Add User Account**

The system should allow users to create new accounts with unique credentials including username, email, and password. Proper validation should be applied to ensure data accuracy and security.

- **User Authentication**

The system should provide login functionality with secure authentication using JWT tokens. Users should be able to securely access their accounts and maintain sessions.

- **Send Messages**

The system should allow authenticated users to send text messages to other users in real-time. Messages should include timestamps and appear instantly in the chat interface.

- **Receive Messages**

The system should deliver incoming messages to recipients instantly without requiring page refresh. Users should see messages appear in real-time as they are sent.

- **View Online Users**

The system should display a list of currently online users, showing presence status through visual indicators. This helps users know who is available for immediate conversation.

- **Message Status Tracking**

The system should show delivery and read status for sent messages, providing confirmation that messages reached their destination and were viewed by recipients.

- **Emoji Support**

The system should allow users to send emojis alongside text messages, enhancing communication and expression. Emoji rendering should work across different devices and browsers.

- **View Message History**

The system should store message history and allow users to view previous conversations when they return to the application.

2.2 Non-Functional Requirements

Non-functional requirements define quality attributes the Real-Time Chat Application must s

- **Performance**: The system should deliver messages instantly with minimal latency and respond quickly to user actions
- **Security**: Only authorized users should access conversations using proper authentication and encrypted connections
- **Usability**: The system should be intuitive and easy to use, allowing users to send messages without difficulty
- **Reliability**: The system should operate consistently and handle errors without message loss
- **Scalability**: The system should handle increasing users and messages without affecting performance
- **Availability**: The system should be available 24/7 with minimal downtime for maintenance

2.3 Requirements Specification

This section specifies the hardware and software needed to run the system.

2.3.1 Minimum Hardware Requirements

- Processor: Intel i5 or above
- RAM: 8 GB (minimum)
- Hard Disk: 50 GB free space
- Network: Stable internet connection

2.3.2 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Programming Language:** JavaScript (ES6+)
- **Frontend Framework:** React
- **Backend Framework:** Node.js with Express.js
- **Real-Time Library:** Socket.IO
- **Database:** MongoDB
- **IDE:** Visual Studio Code
- **Browser:** Chrome, Firefox, Safari, or Edge (latest versions)

3.Technology Description

3.1 Programming Language

3.1.1 Introduction to JavaScript

JavaScript is a high-level, dynamic programming language used extensively for web development. It enables interactive and responsive user interfaces and is supported by all modern web browsers. In the Real-Time Chat Application, JavaScript is used for both frontend and backend development through Node.js.

3.2 Framework / Libraries

3.2.1 Introduction to React

React is a JavaScript library developed by Facebook for building user interfaces. It uses a component-based architecture that makes it easy to create reusable UI elements. In this project, React is used to design the chat interface, message components, and user interface elements. It provides efficient rendering and state management for the application.

3.2.2 Introduction to Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run on the server side. It uses an event-driven, non-blocking I/O model that makes it efficient for real-time applications. In this project, Node.js powers the backend server that handles user authentication, message routing, and database operations.

3.2.3 Introduction to Socket.IO

Socket.IO is a JavaScript library that enables real-time, bidirectional communication between web clients and servers. It uses WebSocket protocol with fallback options for older browsers. In this project, Socket.IO handles the real-time messaging functionality, allowing instant message delivery without page refresh. It manages connections, disconnections, and message broadcasting between users.

3.3 Database Technology

The Real-Time Chat Application uses MongoDB, a NoSQL database, for storing user information and message data. MongoDB stores data in flexible JSON-like documents, making it easy to work with JavaScript applications. It provides high performance for read and write operations, which is essential for real-time messaging.

3.3.1 MongoDB

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents. It provides high performance, scalability, and flexibility for storing chat data. In the Real-Time Chat Application, MongoDB stores user profiles, authentication data, and message history. It supports efficient querying and indexing for fast message retrieval.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code is used as the development environment for the Real-Time Chat Application. It is a lightweight code editor that supports JavaScript, React, and Node.js development through extensions. VS Code provides features such as syntax highlighting, code completion, debugging support, and Git integration, which help improve coding efficiency.

4.Database Design

4.1 Database Collections

The Real-Time Chat Application uses MongoDB collections to store data in a structured manner. The database consists of two main collections: **Users** and **Messages**.

- The **Users** collection stores user account information including credentials and profile data
- The **Messages** collection stores chat messages with metadata

4.2 Collection Structure

4.2.1 Users Collection

Table 1: Users Collection Structure

Field Name	Data Type	Description	Constraint
_id	ObjectId	Unique User ID	Primary Key
username	String	User's Username	Unique, Not Null
email	String	User's Email	Unique, Not Null
password	String	Hashed Password	Not Null
online	Boolean	Online Status	Default: false
lastSeen	Date	Last Activity Time	Not Null

4.2.2 Messages Collection

Table 2: Messages Collection Structure

Field Name	Data Type	Description	Constraint
_id	ObjectId	Message ID	Primary Key
senderId	ObjectId	Sender User ID	Not Null
receiverId	ObjectId	Receiver User ID	Not Null
message	String	Message Content	Not Null
timestamp	Date	Send Time	Not Null
delivered	Boolean	Delivery Status	Default: false
read	Boolean	Read Status	Default: false

4.3 Keys and Constraints

Keys and constraints maintain data integrity and ensure accurate data storage:

- **Primary Keys:** _id fields in both collections
- **Unique Constraints:** username and email fields must be unique
- **Not Null Constraints:** Required fields cannot be left empty
- **Indexing:** Fields used in queries are indexed for faster retrieval

4.4 Collection Relationships

The Users and Messages collections are related through user ID references:

- Each message references its sender and receiver through user IDs
- One user can have multiple messages as sender or receiver
- This represents a one-to-many relationship between Users and Messages

5.System Design

5.1 System Architecture

The Real-Time Chat Application follows a **three-layer architecture**:

1. **Presentation Layer (User Interface)**
    - Developed using React components
    - Allows users to interact with the chat system
    - Handles user input and displays messages
2. **Application Layer (Business Logic)**
    - Processes user inputs and authentication
    - Manages WebSocket connections
    - Routes messages between users
    - Updates user presence status
3. **Data Layer (Database)**
    - Stores user profiles and message data
    - Managed using MongoDB
    - Provides data persistence and retrieval

5.2 Modules Description

The system is divided into the following modules:

1. **User Authentication Module**
    - Handles user registration and login
    - Manages JWT token generation and validation
    - Secures user credentials
2. **Chat Module**
    - Manages message sending and receiving
    - Handles real-time WebSocket communication
    - Displays messages in chat interface
3. **Presence Module**
    - Tracks user online/offline status
    - Updates last seen timestamps
    - Displays online users list
4. **Message Status Module**
    - Updates delivery confirmation
    - Tracks read status of messages
    - Shows status indicators in UI
5. **Emoji Module**
    - Processes emoji input
    - Converts emoji codes to symbols
    - Renders emojis in messages

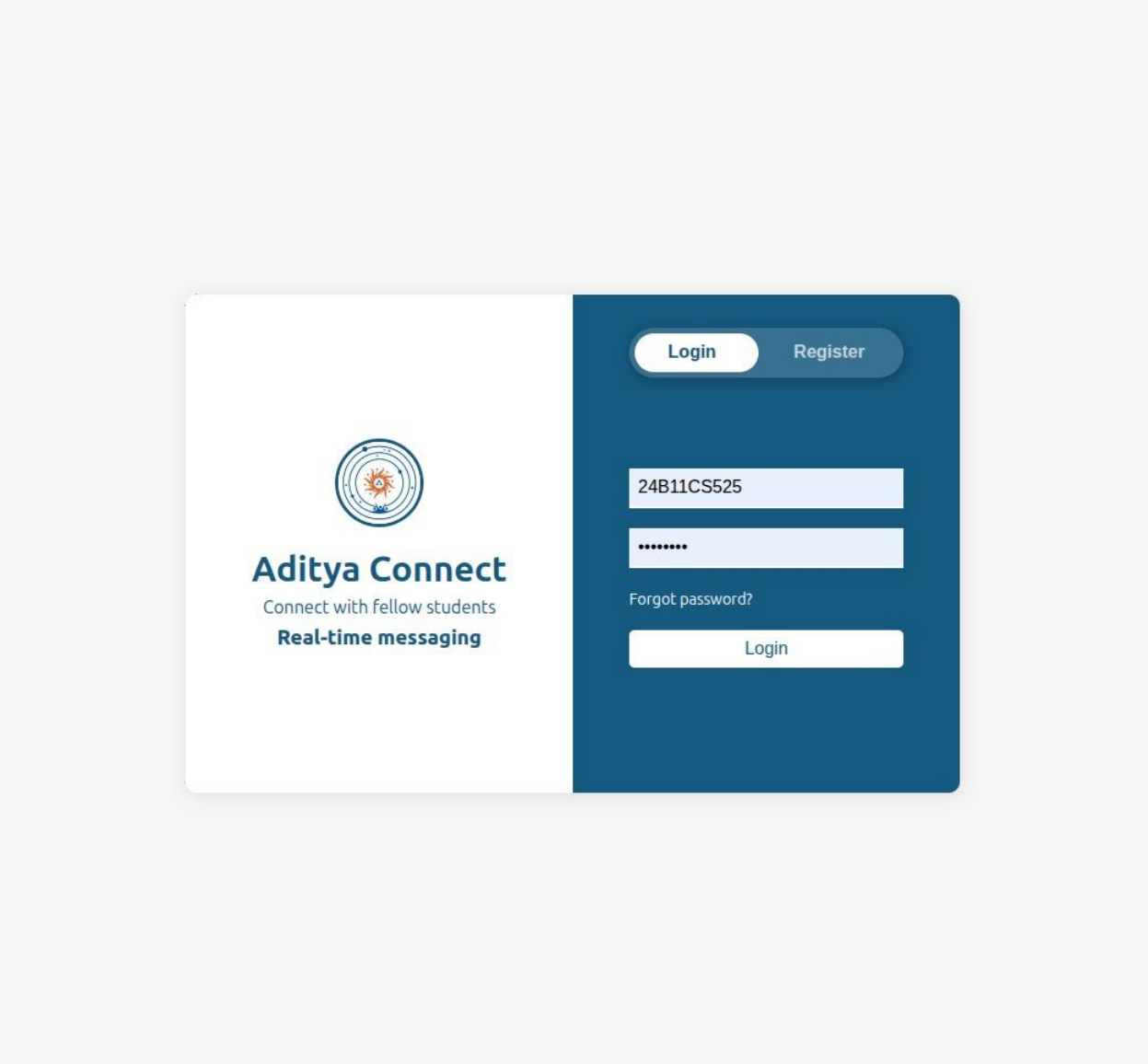
6. **Database Module**
    - Connects to MongoDB
    - Executes CRUD operations
    - Manages data persistence

6. System Implementation

The application implements the following key features:

Login Screen

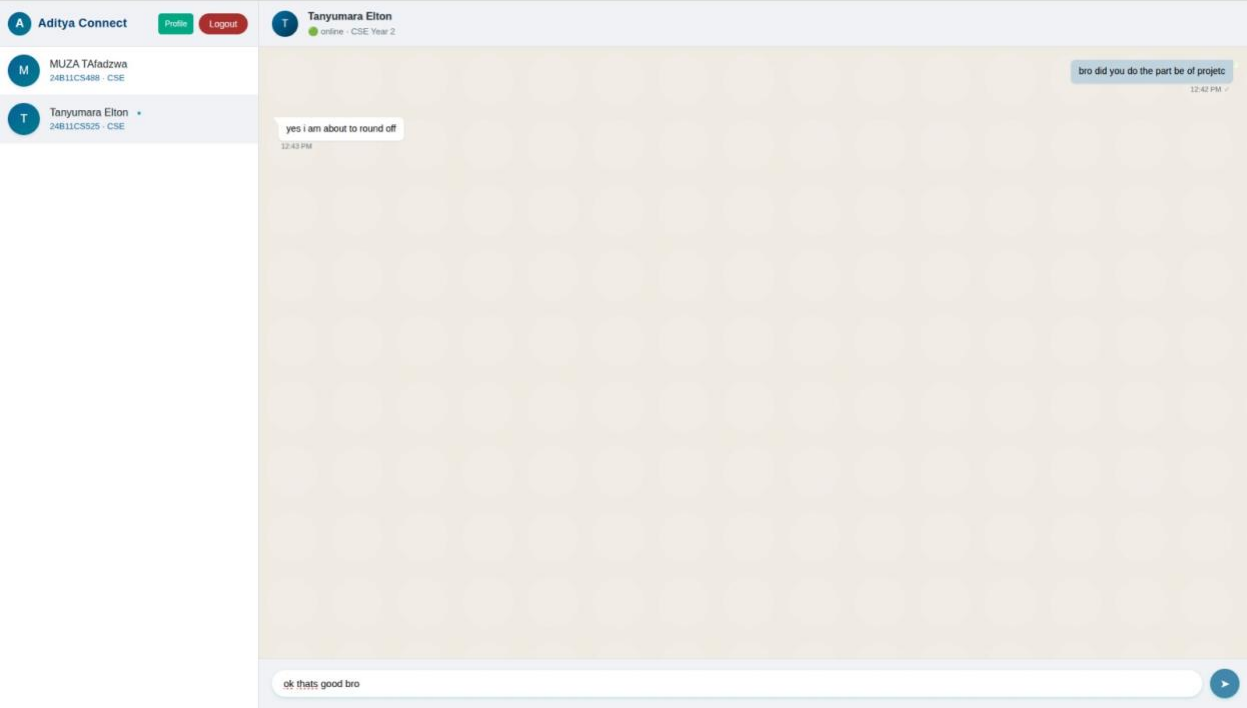
Displays the login interface where users enter username/email and password. Includes registration option for new users.



1.Figure log in

Chat Interface

Provides the main chat window showing conversation history, message input area, and online users list.

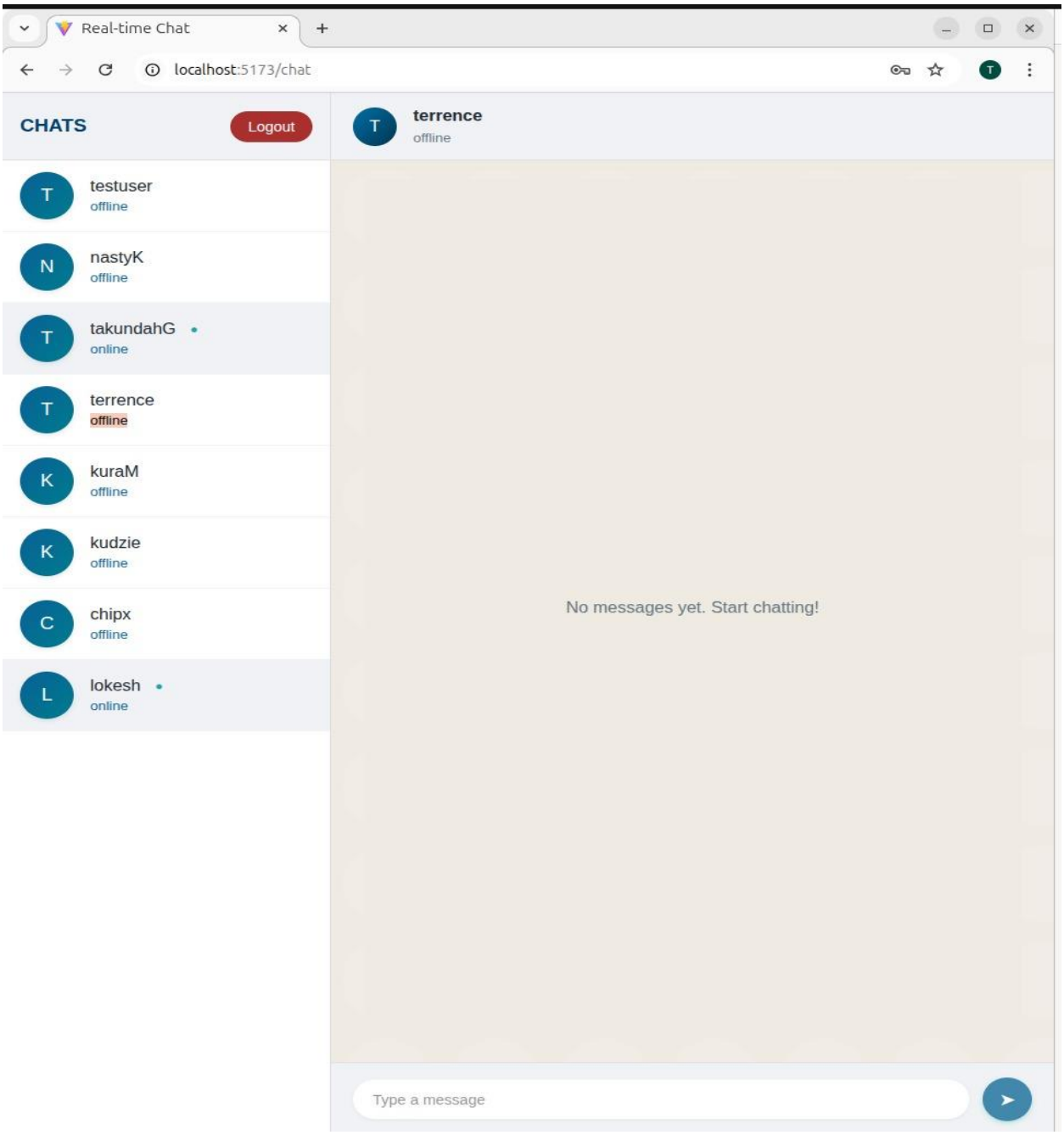


2.Figure Chat

Message Sending & Online Users

Shows the form used to compose and send new messages with emoji picker and send button.

Displays a sidebar showing currently online users with presence indicators.

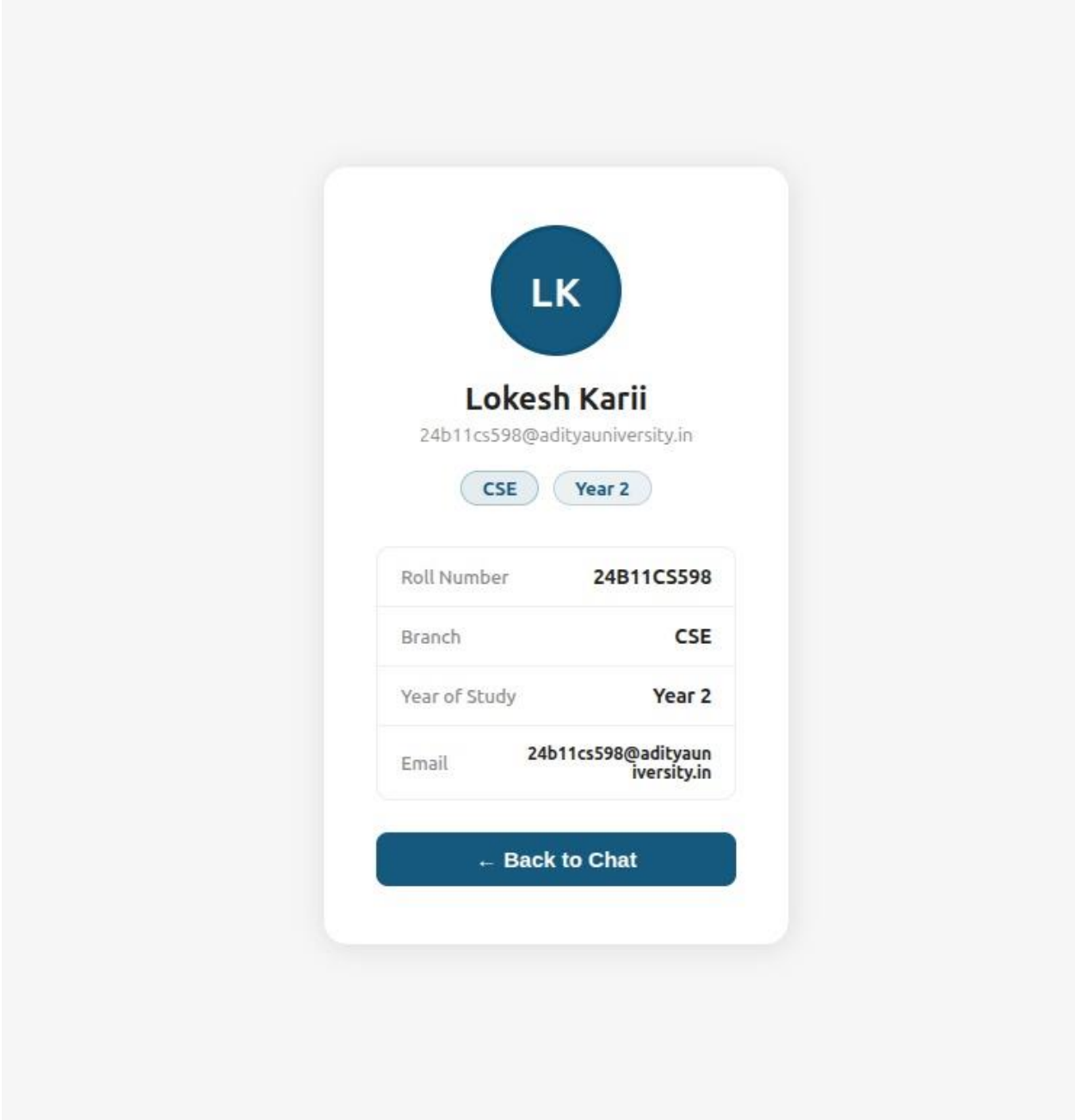


3.Figure Message Sending and Online User

Message Status & User Profile

Shows delivery and read confirmation indicators next to sent messages.

Displays user profile information including username, email, and last seen status.



4.Figure User profile

7. Conclusion

The Real-Time Chat Application has been successfully developed to enable instant messaging between users. The system replaces delayed messaging with real-time communication, reducing delays and improving user experience.

The application provides essential functionalities including user authentication, one-to-one messaging, emoji support, presence detection, and message status tracking. By using technologies like React, Node.js, Socket.IO, and MongoDB, the system ensures reliable performance, secure communication, and efficient data handling.

Overall, the system improves communication efficiency and provides a scalable solution that can be enhanced with additional features such as group chats, file sharing, and video calling.

8. FUTURE ENHANCEMENTS

The Real-Time Chat Application developed in this project provides basic functionalities for messaging. However, the system can be further enhanced by adding advanced features to improve its functionality and usability.

Some possible future enhancements include:

- **Group Chat:**
  Adding support for group conversations with multiple participants
- **File Sharing:**
  Integrating file upload and download capabilities for documents and media
- **Video/Audio Calling:**
  Adding real-time voice and video communication features
- **End-to-End Encryption:**
  Implementing advanced security with encrypted message transmission
- **Message Reactions:**
  Allowing users to react to messages with emojis
- **Dark Mode:**
  Adding theme switching for better user experience

These enhancements can make the system more scalable, secure, and suitable for real-time organizational use.

9. REFERENCES

The following references were used during the development of the Real-Time Chat Application:

1. **MongoDB Documentation**, https://docs.mongodb.com
2. **Express.js Documentation**, https://expressjs.com
3. **React Documentation**, https://reactjs.org
4. **Node.js Documentation**, https://nodejs.org
5. **Socket.IO Documentation**, https://socket.io
6. **MERN Stack Tutorial – GeeksforGeeks**, https://www.geeksforgeeks.org/mern-stack/
7. **WebSocket Protocol Specification**, RFC 6455
8. **JavaScript: The Definitive Guide**, David Flanagan, O'Reilly Media

APPENDIX

A.1 Sample Code

Server Connection Code:

The following code establishes a connection between the Node.js server and the MongoDB database.

server.js:

```
const express = require('express');
const mongoose = require('mongoose');
const socketIo = require('socket.io');

const app = express();
const PORT = process.env.PORT || 3000;

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/chatdb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

// Socket.IO setup
const server = app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

const io = socketIo(server);

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
  });
});
```

Message Sending Code:

This code is used to save a new message to the database and emit it to the recipient.

messageController.js:

```
const Message = require('../models/Message');
```

```
exports.sendMessage = async (req, res) => {
  try {
    const { senderId, receiverId, content } = req.body;

    const message = new Message({
      senderId,
      receiverId,
      content,
      timestamp: new Date(),
      status: 'sent'
    });

    await message.save();

    // Emit message to recipient via Socket.IO
    io.to(receiverId).emit('newMessage', message);

    res.status(201).json({ message: 'Message sent successfully'
});
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

A.2 API Endpoints

User Authentication Endpoints:

```
POST /api/auth/register - User registration
POST /api/auth/login - User login
GET /api/auth/logout - User logout
```

Message Endpoints:

```
POST /api/messages/send - Send new message
GET /api/messages/:userId - Get message history
PUT /api/messages/:id/read - Mark message as read
```

User Endpoints:

```
GET /api/users - Get all users
GET /api/users/:id - Get user by ID
PUT /api/users/:id/status - Update user status
```